

Exp: 6 - Write the python program for Vacuum Cleaner Problem.

Aim:

To write a python program for vacuum cleaner problem.

Algorithm:

Step 1: Create a data structure to represent the state of the problem, including the current position of the vacuum cleaner and the state of each square in the room (clean or dirty).

Step 2: Create a data structure to represent the state space of the problem, including the initial state, the goal state, and the possible transitions between states.

Step 3: Use a search algorithm, such as depth-first search or breadth-first search, to explore the state space and find a solution.

Step 4: At each step of the search, generate all possible successor states from the current state by applying one of the allowed actions (move up, down, left, right, or clean).

Step 5: Check if the successor state is valid (i.e., the vacuum cleaner cannot move outside the room, and it can only clean dirty squares).

Step 6: If the successor state is valid, add it to the search tree and continue the search.

Step 7: If the successor state is the goal state, return the path from the initial state to the goal state.

Step 8: If there are no more possible successor states to explore, backtrack to the previous state and try another possible action.

Program:

```
from typing import List, Tuple
```

```
def get_successors(state):  
    position, grid = state  
    successors = []
```

```
    x, y = position
```

```
    for dx, dy in [(0,1), (0,-1), (1,0), (-1,0)]:
```

```

    new_x = x + dx
    new_y = y + dy

    if 0 <= new_x < len(grid) and 0 <= new_y < len(grid[0]):

        new_grid = [row[:] for row in grid]

        if new_grid[new_x][new_y] == 1:
            new_grid[new_x][new_y] = 0

        successors.append(((new_x, new_y), new_grid))

    return successors

def depth_first_search(state, visited):

    if all(all(cell == 0 for cell in row) for row in state[1]):
        return state

    visited.add(str(state))

    for successor in get_successors(state):

        if str(successor) not in visited:

            result = depth_first_search(successor, visited)

            if result:
                return result

    return None

grid = [
    [0, 1, 1, 1],
    [0, 0, 1, 0],
    [1, 0, 0, 1],
    [1, 1, 1, 0]
]

```

```

initial_state = ((0, 0), grid)

result = depth_first_search(initial_state, set())

if result:

    print("The room can be cleaned.\n")

    print("Final Grid:")

    final_grid = result[1]

    for row in final_grid:
        print(row)

else:
    print("The room cannot be cleaned.")

```

Output:

```

_ _ _ _ _
The room can be cleaned.

Final Grid:|
[0, 0, 0, 0]
[0, 0, 0, 0]
[0, 0, 0, 0]
[0, 0, 0, 0]

```

Result:

Hence, the simple python program for Vacuum Cleaner Problem was done.